

Homework 02-2: due 2024/04/08 23:59 (70%)

- Tutorial : <https://machinelearningmastery.com/pytorch-tutorial-develop-deep-learning-models/>
- After you go through the tutorials, you should be able to work on this assignment.
- Please answer the following questions and work directly on this jupyter notebook.
- Make sure the code can be run and show the result and figures properly.
- Please write down your observation with markdown in this notebook briefly.

You will train a regression model in this part. The data is **Concrete Compressive Strength Dataset**, you can find the details of each column at <https://archive.ics.uci.edu/ml/datasets/concrete+compressive+strength>. In this part, please predict the **strength of concrete**.

```
In [1]: # Import necessary modules
%matplotlib inline
import pandas as pd
import numpy as np
import random
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
import matplotlib.pyplot as plt
from tqdm import tqdm
from sklearn.preprocessing import MinMaxScaler
```

```
In [2]: # For reproduce the result
torch.manual_seed(1220)
random.seed(1220)
np.random.seed(1220)
```

```
In [3]: device = (
    "cuda"
    if torch.cuda.is_available()
    else "mps"
    if torch.backends.mps.is_available()
    else "cpu"
)
print(f"Using {device} device")
```

Using cuda device

1. Define the model and dataset (10%)

1.1 Please follow the Step 2 from the tutorial to create a class ***ConcreteDataset*** to load the data you need and also do the ***Min-Max scaling*** to the ***feature and label***. (5%)

Define transformation

```
In [4]: class Min_Max_Scaler:
    def __init__(self):
        None

    def fit(self, data):
        self.min = np.min(data, axis=0) #axis=0 → operate column-wise.Stores values into self.min and self.max
        self.max = np.max(data, axis=0)

    def transform(self, data):
        scaling_data = data.copy() # Creates a copy of the input data to avoid modifying the original data
        for i in range(data.shape[1]): #data.shape[1] gives the number of columns in the data.
            scaling_data[:, i] = 2 * ((data[:, i] - self.min[i]) / (self.max[i] - self.min[i])) - 1
        return scaling_data # This transformation scales each column of the data to the range [-1, 1].

    def fit_transform(self, data): #
        self.fit(data)
        return self.transform(data)

    def y_inverse_transform(self, scaling_y):
        return (scaling_y + 1) * (self.max[-1] - self.min[-1]) / 2 + self.min[-1]
```

```
In [5]: """
A skeleton of a custom Dataset class is provided below.
Once loaded, PyTorch provides the DataLoader class to navigate a
Dataset instance during the training and evaluation of your model.
"""

class ConcreteDataset(Dataset):
    # load the dataset
    def __init__(self, data):
        # store the inputs and outputs
        self.X = torch.tensor(data[:, :-1]).to(torch.float32)
        self.y = torch.tensor(data[:, -1]).to(torch.float32) #Extracts all columns except the last one (features)

    # number of rows in the dataset
    def __len__(self):
        return len(self.X) # Returns the number of samples in the dataset. This is needed by PyTorch to know the

    # get a row at an index
    def __getitem__(self, idx):
        return self.X[idx], self.y[idx] # . It allows PyTorch to access individual samples from your dataset whe
```

1.2 By following the tutorial, try to create a class ***MLP(Neural Network)*** with ***three hidden layers*** as your network architecture. Also, for the convenience of implementation, please set the numbers of hidden nodes and the activation functions as input variables to the forward functions.(5%)

Build model

```
In [ ]: class MLP_1(nn.Module): #“I want my MLP class to be based on PyTorch’s built-in nn.Module class.”
    def __init__(self):
        super().__init__() #Think of super().__init__() like calling your parent’s setup function before you a
        self.net = nn.Sequential(
            nn.Linear(8, 256),
            nn.Tanh(),
            nn.Linear(256, 64),
            nn.Tanh(),
            nn.Linear(64, 8),
            nn.Tanh(),
            nn.Linear(8, 1),
            nn.Tanh(),
        )

    def forward(self, X):
        outputs = self.net(X)
        return outputs
```

```
In [24]: class MLP_2(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(8, 64),
            nn.Tanh(),
            nn.Linear(64, 16),
            nn.Tanh(),
            nn.Linear(16, 8),
            nn.Tanh(),
            nn.Linear(8, 1),
        )

    def forward(self, X):
        outputs = self.net(X)
        return outputs
```

```
In [12]: """another version with flexible inputs variables"""
import torch
import torch.nn as nn

class FlexibleMLP(nn.Module):
    def __init__(self, input_dim=8, output_dim=1, hidden_sizes=[256, 64, 8],
                 activation_fn=nn.Tanh, output_activation=None):
        super().__init__()

        layers = []
        in_dim = input_dim
        output_activation= output_activation if output_activation is not None else nn.Identity # Default to id

        for h in hidden_sizes:
            layers.append(nn.Linear(in_dim, h))
            layers.append(activation_fn())
            in_dim = h

        layers.append(nn.Linear(in_dim, output_dim))
```

```

        if output_activation is not None:
            layers.append(output_activation()) # Add output activation if provided

        self.net = nn.Sequential(*layers)

    def forward(self, X):
        return self.net(X)

```

```

In [ ]: # Example usage (skip this part if you want to use the FlexibleMLP class directly)

# Use like any PyTorch model
model = FlexibleMLP(
    input_dim=8,
    output_dim=1,
    hidden_sizes=[256, 128, 64],
    activation_fn=nn.Tanh # or nn.ReLU, nn.LeakyReLU
)

# Use like any PyTorch model
output = model(torch.randn(32, 8)) # batch of 32 samples, each with 8 features

# save and load Example

# Save model
torch.save(model.state_dict(), 'flexible_mlp.pth') # Save the model's weights Creates a .pth file

# Load model later
model_loaded = FlexibleMLP(input_dim=8, output_dim=1, hidden_sizes=[256, 128, 64], activation_fn=nn.Tanh) # Re
model_loaded.load_state_dict(torch.load('flexible_mlp.pth')) # Load saved weights into the new model Fills in
model_loaded.eval() # Switch to inference mode Use before making predictions #Puts the model into "evaluation

```

C:\Users\USER\AppData\Local\Temp\ipykernel_11664\1206862356.py:21: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowed by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```

    model_loaded.load_state_dict(torch.load('flexible_mlp.pth')) # Load saved weights into the new model Fills in
the trained values

```

```

Out[ ]: FlexibleMLP(
  (net): Sequential(
    (0): Linear(in_features=8, out_features=256, bias=True)
    (1): Tanh()
    (2): Linear(in_features=256, out_features=128, bias=True)
    (3): Tanh()
    (4): Linear(in_features=128, out_features=64, bias=True)
    (5): Tanh()
    (6): Linear(in_features=64, out_features=1, bias=True)
    (7): Identity()
  )
)

```

if the model is not stored in self, so:

You can't use `.parameters()` on self to get trainable weights

You can't save or load the model state easily using `torch.save()` or `model.state_dict()`

To fix that, you could build `self.net = nn.Sequential(...)` inside **init** after receiving configuration — or pass configuration at construction.

Typical Pattern in Code

```

nn.Linear(8, 256), # Linear layer
nn.Tanh(),         # Non-linear activation

```

This **pair** (linear + activation) is what actually gives the network learning power.

✓ Summary

Component	Type	Purpose
<code>nn.Linear(...)</code>	Layer	Applies a linear transformation
<code>nn.Tanh()</code>	Activation Function	Adds non-linearity to the model
Together	Layer block	Forms a powerful unit for learning patterns

✗ `nn.Linear(8, 256)` is **not** an activation function.

It is a **fully connected (dense) layer**, also called a **linear transformation layer** in PyTorch.

Step 1: Understand Your Problem Type

Problem Type	Typical Output
Regression	A single number (e.g., concrete strength) → output size = 1
Binary Classification	Probability of class (0 or 1) → output size = 1 with <code>Sigmoid()</code>
Multi-class Classification	Predict 1 of N classes → output size = N with <code>Softmax()</code>

Step 2: Know Your Input Size

- Count the number of **input features** (columns in your data).
- Example: If your dataset has 8 features (like concrete dataset), then:

```
nn.Linear(8, ?)
```

Step 3: Decide on Architecture

Rule of thumb: start **simple**, then add complexity if needed.

☆☆ A. Number of Hidden Layers

Layer Depth	When to Use
1 hidden layer	Usually enough for simple problems (shallow MLPs)
2–3 hidden layers	Good for most moderate problems
4+ hidden layers	For very complex tasks (image, NLP, etc.) or big datasets

✓ For tabular data (like the concrete dataset), **2 to 3 hidden layers** are usually enough.

☆☆ B. Number of Neurons (Width)

No fixed rule, but here's a good **starting pattern**:

```
[input_dim] → [4×input_dim] → [2×input_dim] → [input_dim] → [1]
```

Example: For 8 input features

```
nn.Linear(8, 32), # 4×8
nn.ReLU(),
nn.Linear(32, 16), # 2×8
nn.ReLU(),
nn.Linear(16, 8), # back to input size
nn.ReLU(),
nn.Linear(8, 1) # output for regression
```

Step 4: Activation Functions

Function	When to Use
<code>ReLU()</code>	Default for hidden layers — fast and works well
<code>Tanh()</code>	Use if your input/output is scaled to [-1, 1]
<code>Sigmoid()</code>	Binary output (probability)
<code>Softmax()</code>	Multi-class classification output

For most tasks:

hidden → ReLU
output → depends on task

Step 5: Tune and Experiment

Try:

- **Start simple** → test performance
- Use **validation loss** to evaluate
- Increase width/depth only if underfitting
- Use **dropout** if overfitting (too good on training, bad on test)

```
nn.Dropout(0.3) # randomly disables 30% of neurons during training
```

✓ General Tips

Strategy	Why
Use powers of 2 (like 32, 64, 128)	More efficient on modern hardware
Fewer layers for small datasets	Avoid overfitting
Normalize your input	Helps training (Min-Max or Z-score)
Use early stopping	Prevents overfitting
Visualize training/validation loss	Helps diagnose under/overfitting

Summary: How to Design MLP

1. **Input size** = number of features
2. **Output size** = based on task (1 for regression)
3. **Hidden layers**: start with 2 or 3
4. **Neurons**: $4 \times \text{input} \rightarrow 2 \times \text{input} \rightarrow \text{input}$
5. **Activations**: ReLU (hidden), Tanh/Sigmoid/None (output)
6. **Tune & validate** with actual performance

✓ Best Practices

1. **Start small**, then increase only if:

- Training loss is still high (underfitting)
- Validation loss is not getting worse (no overfitting)

2. Use regularization if going big:

- Add **Dropout** between layers:

```
nn.Dropout(0.3)
```

- Add **weight decay** (L2 penalty) in your optimizer.

3. Use early stopping during training

- Stop training when validation loss stops improving.

Example: Small vs. Large Design

Design	Layers	Good For
Small	8 → 32 → 16 → 8 → 1	Small datasets, fast prototyping
Medium	8 → 64 → 32 → 16 → 1	Medium-sized tabular data
Large	8 → 128 → 64 → 32 → 1	Big datasets or highly nonlinear patterns
Very large	8 → 256 → 128 → 64 → 1	Only if you have a lot of data (10000+ samples)

Final Tip

You can use **validation curves** to guide your architecture choice:

- Try plotting **model size vs. performance**
- Start small, and increase **only if** you see **consistent improvement**

2. Train the model (60%)

2.1 Please load the ***train.csv/ validation.csv*** in `./data`, and turn them into dataloader with batch size 64 and determine whether shuffle or not. (5%)

Load data

```
In [13]: train_data = np.array(pd.read_csv("./data/train.csv"))
test_data = np.array(pd.read_csv("./data/test.csv"))
valid_data = np.array(pd.read_csv("./data/validation.csv"))

scaler = Min_Max_Scaler()

train_data_scaled = scaler.fit_transform(train_data) # You must only fit the scaler on the training data.
test_data_scaled = scaler.transform(test_data) # dont fit test and validation data prevent data leakage
valid_data_scaled = scaler.transform(valid_data) # The scaler is fitted on the training data and then used on the test and validation data

train_data_loader = DataLoader(ConcreteDataset(train_data_scaled), batch_size=64, shuffle=True)
test_data_loader = DataLoader(ConcreteDataset(test_data_scaled))
valid_data_loader = DataLoader(ConcreteDataset(valid_data_scaled), batch_size=64, shuffle=True)
```

Batch size

Rule of Thumb

Start with 32 or 64 If you have more memory and training is smooth, try **doubling** it If you get out-of-memory or poor performance, **halve** it

✓ Summary Table

Factor	Effect on Batch Size
More memory (GPU)	Allows larger batches
Larger model	Needs smaller batches
Smaller dataset	Use larger batches
Overfitting risk	Use smaller batches
Smoother training curve	Use larger batches

Batchnorm eval and train

✓ `model.train()` vs `model.eval()`

Mode	What it means	What changes
<code>model.train()</code>	Training mode	Enables Dropout , uses batch stats in BatchNorm
<code>model.eval()</code>	Evaluation mode	Disables Dropout , uses saved stats in BatchNorm

1. `model.train()` : Training Mode

This is the **default** mode. You call this when training your model.

`model.train()`

✓ What happens:

- **Dropout** layers randomly drop neurons (for regularization)
- **BatchNorm** layers compute **mean/variance from the current mini-batch**

This keeps your model flexible and learning properly.

2. `model.eval()` : Evaluation Mode

You call this before:

- **Validation**
- **Testing**
- **Saving predictions**

```
model.eval()
```

✓ What changes:

- **Dropout** is turned OFF → no random behavior
- **BatchNorm** uses **saved (running average)** of mean/variance from training

This makes your results **stable and reproducible** — exactly what you want when measuring accuracy or making predictions.

Data loader parameters

Parameter	Use When...
shuffle=True	Training data, to randomize batches
shuffle=False	Validation or test data
drop_last=True	You want all batches the same size (e.g., for BatchNorm)
num_workers=0	Simpler code/debugging; use on Windows
num_workers=2+	You want faster data loading (on Linux/macOS)
pin_memory=True	You are training on GPU and want faster data transfer

2.2 Create two MLP model from the table below and ***print the model*** (10%):

	Hidden Layer 1	Hidden Layer 2	Hidden Layer 3	Activation in each hidden nodes	Activation for output
Model1	256	64	8	Tanh	Tanh
Model1	64	16	8	Tanh	Identity

```
In [ ]: # default model
model_1 = MLP_1().to(device)
model_2 = MLP_2().to(device)
print("Model 1:\n", model_1, "\n\nModel 1:\n", model_2)
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[8], line 1
----> 1 model_1 = MLP_1().to(device)
      2 model_2 = MLP_2().to(device)
      3 print("Model 1:\n", model_1, "\n\nModel 1:\n", model_2)

NameError: name 'MLP_1' is not defined
```

```
In [40]: # Flexible model
model_1 = FlexibleMLP(
    input_dim=8,
    output_dim=1,
    hidden_sizes=[256, 64, 8],
    activation_fn=nn.Tanh,
    output_activation=nn.Tanh
).to(device)

model_2 = FlexibleMLP(
    input_dim=8,
    output_dim=1,
    hidden_sizes=[64, 16, 8],
    activation_fn=nn.Tanh,
    output_activation=nn.Identity # Identity, i.e., no activation
).to(device)

print("Model 1:\n", model_1, "\n\nModel 1:\n", model_2)
```

```

Model 1:
FlexibleMLP(
  (net): Sequential(
    (0): Linear(in_features=8, out_features=256, bias=True)
    (1): Tanh()
    (2): Linear(in_features=256, out_features=64, bias=True)
    (3): Tanh()
    (4): Linear(in_features=64, out_features=8, bias=True)
    (5): Tanh()
    (6): Linear(in_features=8, out_features=1, bias=True)
    (7): Tanh()
  )
)

```

```

Model 1:
FlexibleMLP(
  (net): Sequential(
    (0): Linear(in_features=8, out_features=64, bias=True)
    (1): Tanh()
    (2): Linear(in_features=64, out_features=16, bias=True)
    (3): Tanh()
    (4): Linear(in_features=16, out_features=8, bias=True)
    (5): Tanh()
    (6): Linear(in_features=8, out_features=1, bias=True)
    (7): Identity()
  )
)

```

to device

Why move only the model to GPU?

Moving the model to GPU allows its parameters to live in GPU memory. But the **data comes in one batch at a time**, so we:

- Move the **model once** with `.to(device)`
- Move the **data batch-by-batch** during training/inference

This saves **GPU memory** and **avoids crashing** if the dataset is large.

2.3 Train the above two models with the same hyperparameters below and do the validation in every epoch. Choose the appropriate type of loss function according to the task. (25%)

(Note. You should record the training/validation loss every epoch)

	Learning rate	epochs	optimizer	momentum
Hyperparameter	0.01	300	SGD	0.9

Define loss funct + optimizer

```

In [41]: loss_fn = nn.MSELoss()
optimizer_1 = torch.optim.SGD(model_1.parameters(), lr=0.01, momentum=0.9)
optimizer_2 = torch.optim.SGD(model_2.parameters(), lr=0.01, momentum=0.9)

```

Define train funct and validation funct

```

In [43]: def train(dataloader, model, loss_fn, optimizer):
    epoch_loss = 0.
    model.train()
    for batch, (X, y) in enumerate(dataloader):  # batch is batch index, (X, y) is the minibatch data
        X, y = X.to(device), y.to(device)  # if your batch_size = 64 and your dataset has 320 samples:
        # You'll run 5 iterations (320 ÷ 64 = 5) Each time, X will be a tensor of shape [64, num_features], and

        pred = model(X)
        loss = loss_fn(pred, y.unsqueeze(1))  # MSE loss function outputs a value that measures the average squ

        optimizer.zero_grad()  # Clears old gradients from the previous step (PyTorch accumulates them by defau
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()  # Converts the scalar tensor (like tensor(0.054)) Into a python float: 0.054

    epoch_loss /= len(dataloader)
    return epoch_loss

def valid(dataloader, model, loss_fn):
    epoch_loss = 0.
    model.eval()  # Switch to evaluation mode. This is important because it changes the behavior of certain
    with torch.no_grad():  # Disables gradient calculation, which reduces memory consumption for computatio

```



```

for X, y in dataloader:
    X, y = X.to(device), y.to(device)  # to set input and output tensors to the same device as the model
    # This ensures that the model can process the data without device mismatch errors.

    pred = model(X)
    loss = loss_fn(pred, y.squeeze(1))
    epoch_loss += loss.item()

epoch_loss /= len(dataloader)
return epoch_loss

```

Run train

△ 小提醒：效能問題 在每次用 `np.append()` 時，它都會建立一個新的陣列並複製原始資料 → 對於大量迴圈（如 300 次 epoch）來說效率不高。

✓ 更好的寫法是用 Python list 收集，再轉成 NumPy 陣列：`train_loss_1, valid_loss_1, train_loss_2, valid_loss_2 = [], [], [], []`

```

for i in loop:
    train_loss_1.append(train(train_data_loader, model_1, loss_fn, optimizer_1))
    valid_loss_1.append(valid(valid_data_loader, model_1, loss_fn))
    train_loss_2.append(train(train_data_loader, model_2, loss_fn, optimizer_2))
    valid_loss_2.append(valid(valid_data_loader, model_2, loss_fn))

```

```

loop.set_description(f"Epoch {i+1}/{epochs}")
loop.set_postfix(
    model_1_train_loss=f"{train_loss_1[-1]:.6f}",
    model_1_valid_loss=f"{valid_loss_1[-1]:.6f}",
    model_2_train_loss=f"{train_loss_2[-1]:.6f}",
    model_2_valid_loss=f"{valid_loss_2[-1]:.6f}"
)

```

```
import numpy as np
```

```

train_loss_1 = np.array(train_loss_1)
valid_loss_1 = np.array(valid_loss_1)
train_loss_2 = np.array(train_loss_2)
valid_loss_2 = np.array(valid_loss_2)

```

Train

```

In [28]: epochs = 300
loop = tqdm(range(epochs))
train_loss_1, valid_loss_1, train_loss_2, valid_loss_2 = np.array([]), np.array([]), np.array([]), np.array([])
for i in loop:
    train_loss_1 = np.append(train_loss_1, train(train_data_loader, model_1, loss_fn, optimizer_1))
    valid_loss_1 = np.append(valid_loss_1, valid(valid_data_loader, model_1, loss_fn))
    train_loss_2 = np.append(train_loss_2, train(train_data_loader, model_2, loss_fn, optimizer_2))
    valid_loss_2 = np.append(valid_loss_2, valid(valid_data_loader, model_2, loss_fn))
    loop.set_description(f"Epoch {i+1}/{epochs}")
    loop.set_postfix(model_1_train_loss=f"{train_loss_1[-1]:.6f}", model_1_valid_loss=f"{valid_loss_1[-1]:.6f}"

```

```

Epoch 300/300: 100%|██████████| 300/300 [00:14<00:00, 20.64it/s, model_1_train_loss=0.026035, model_1_valid_loss=0.066412, model_2_train_loss=0.028963, model_2_valid_loss=0.079764]

```

```

In [32]: epochs = 300
train_loss_1, valid_loss_1, train_loss_2, valid_loss_2 = np.array([]), np.array([]), np.array([]), np.array([])

for i in range(epochs):
    tl1 = train(train_data_loader, model_1, loss_fn, optimizer_1)
    vl1 = valid(valid_data_loader, model_1, loss_fn)
    tl2 = train(train_data_loader, model_2, loss_fn, optimizer_2)
    vl2 = valid(valid_data_loader, model_2, loss_fn)

    train_loss_1 = np.append(train_loss_1, tl1)
    valid_loss_1 = np.append(valid_loss_1, vl1)
    train_loss_2 = np.append(train_loss_2, tl2)
    valid_loss_2 = np.append(valid_loss_2, vl2)

    print(f"Epoch {i+1:03}/{epochs}: "
          f"Model 1 - Train Loss: {tl1:.6f}, Valid Loss: {vl1:.6f} | "
          f"Model 2 - Train Loss: {tl2:.6f}, Valid Loss: {vl2:.6f}")

```

```

Epoch 001/300: Model 1 - Train Loss: 0.170300, Valid Loss: 0.144343 | Model 2 - Train Loss: 0.207187, Valid Loss: 0.185239
Epoch 002/300: Model 1 - Train Loss: 0.131582, Valid Loss: 0.112934 | Model 2 - Train Loss: 0.156538, Valid Loss: 0.123358
Epoch 003/300: Model 1 - Train Loss: 0.103417, Valid Loss: 0.100188 | Model 2 - Train Loss: 0.130060, Valid Loss: 0.132257
Epoch 004/300: Model 1 - Train Loss: 0.087171, Valid Loss: 0.121375 | Model 2 - Train Loss: 0.109328, Valid Loss: 0.121375

```

: 0.113849
Epoch 005/300: Model 1 - Train Loss: 0.077451, Valid Loss: 0.116026 | Model 2 - Train Loss: 0.096307, Valid Loss : 0.124739
Epoch 006/300: Model 1 - Train Loss: 0.074844, Valid Loss: 0.110060 | Model 2 - Train Loss: 0.088606, Valid Loss : 0.111563
Epoch 007/300: Model 1 - Train Loss: 0.072706, Valid Loss: 0.109292 | Model 2 - Train Loss: 0.082383, Valid Loss : 0.115804
Epoch 008/300: Model 1 - Train Loss: 0.072002, Valid Loss: 0.114896 | Model 2 - Train Loss: 0.078365, Valid Loss : 0.111596
Epoch 009/300: Model 1 - Train Loss: 0.072378, Valid Loss: 0.078782 | Model 2 - Train Loss: 0.075678, Valid Loss : 0.122309
Epoch 010/300: Model 1 - Train Loss: 0.073056, Valid Loss: 0.097780 | Model 2 - Train Loss: 0.074411, Valid Loss : 0.121849
Epoch 011/300: Model 1 - Train Loss: 0.070574, Valid Loss: 0.117599 | Model 2 - Train Loss: 0.073029, Valid Loss : 0.103628
Epoch 012/300: Model 1 - Train Loss: 0.069683, Valid Loss: 0.091659 | Model 2 - Train Loss: 0.071763, Valid Loss : 0.117008
Epoch 013/300: Model 1 - Train Loss: 0.069320, Valid Loss: 0.110023 | Model 2 - Train Loss: 0.071971, Valid Loss : 0.106038
Epoch 014/300: Model 1 - Train Loss: 0.069995, Valid Loss: 0.091177 | Model 2 - Train Loss: 0.072058, Valid Loss : 0.084668
Epoch 015/300: Model 1 - Train Loss: 0.068298, Valid Loss: 0.077803 | Model 2 - Train Loss: 0.070504, Valid Loss : 0.099100
Epoch 016/300: Model 1 - Train Loss: 0.069146, Valid Loss: 0.079936 | Model 2 - Train Loss: 0.069868, Valid Loss : 0.094341
Epoch 017/300: Model 1 - Train Loss: 0.068663, Valid Loss: 0.086748 | Model 2 - Train Loss: 0.069690, Valid Loss : 0.097399
Epoch 018/300: Model 1 - Train Loss: 0.067905, Valid Loss: 0.097843 | Model 2 - Train Loss: 0.070020, Valid Loss : 0.093268
Epoch 019/300: Model 1 - Train Loss: 0.068344, Valid Loss: 0.074754 | Model 2 - Train Loss: 0.069518, Valid Loss : 0.088698
Epoch 020/300: Model 1 - Train Loss: 0.068749, Valid Loss: 0.062123 | Model 2 - Train Loss: 0.069080, Valid Loss : 0.089670
Epoch 021/300: Model 1 - Train Loss: 0.068966, Valid Loss: 0.069189 | Model 2 - Train Loss: 0.068982, Valid Loss : 0.089119
Epoch 022/300: Model 1 - Train Loss: 0.066330, Valid Loss: 0.074739 | Model 2 - Train Loss: 0.068521, Valid Loss : 0.091969
Epoch 023/300: Model 1 - Train Loss: 0.065451, Valid Loss: 0.069318 | Model 2 - Train Loss: 0.068409, Valid Loss : 0.077670
Epoch 024/300: Model 1 - Train Loss: 0.065114, Valid Loss: 0.067955 | Model 2 - Train Loss: 0.068192, Valid Loss : 0.076991
Epoch 025/300: Model 1 - Train Loss: 0.064270, Valid Loss: 0.065986 | Model 2 - Train Loss: 0.067206, Valid Loss : 0.081236
Epoch 026/300: Model 1 - Train Loss: 0.064370, Valid Loss: 0.065498 | Model 2 - Train Loss: 0.067596, Valid Loss : 0.087255
Epoch 027/300: Model 1 - Train Loss: 0.063431, Valid Loss: 0.065462 | Model 2 - Train Loss: 0.067657, Valid Loss : 0.084621
Epoch 028/300: Model 1 - Train Loss: 0.061996, Valid Loss: 0.075160 | Model 2 - Train Loss: 0.068188, Valid Loss : 0.068310
Epoch 029/300: Model 1 - Train Loss: 0.061427, Valid Loss: 0.060712 | Model 2 - Train Loss: 0.066807, Valid Loss : 0.072641
Epoch 030/300: Model 1 - Train Loss: 0.059788, Valid Loss: 0.070596 | Model 2 - Train Loss: 0.065425, Valid Loss : 0.085152
Epoch 031/300: Model 1 - Train Loss: 0.059795, Valid Loss: 0.059514 | Model 2 - Train Loss: 0.065776, Valid Loss : 0.068551
Epoch 032/300: Model 1 - Train Loss: 0.059278, Valid Loss: 0.064006 | Model 2 - Train Loss: 0.064742, Valid Loss : 0.075460
Epoch 033/300: Model 1 - Train Loss: 0.059252, Valid Loss: 0.064466 | Model 2 - Train Loss: 0.064717, Valid Loss : 0.070919
Epoch 034/300: Model 1 - Train Loss: 0.057444, Valid Loss: 0.056425 | Model 2 - Train Loss: 0.063791, Valid Loss : 0.070621
Epoch 035/300: Model 1 - Train Loss: 0.056343, Valid Loss: 0.054126 | Model 2 - Train Loss: 0.063466, Valid Loss : 0.075901
Epoch 036/300: Model 1 - Train Loss: 0.055295, Valid Loss: 0.053870 | Model 2 - Train Loss: 0.063163, Valid Loss : 0.068089
Epoch 037/300: Model 1 - Train Loss: 0.053987, Valid Loss: 0.052718 | Model 2 - Train Loss: 0.062068, Valid Loss : 0.055800
Epoch 038/300: Model 1 - Train Loss: 0.052875, Valid Loss: 0.055371 | Model 2 - Train Loss: 0.061924, Valid Loss : 0.059964
Epoch 039/300: Model 1 - Train Loss: 0.051557, Valid Loss: 0.051974 | Model 2 - Train Loss: 0.060425, Valid Loss : 0.063389
Epoch 040/300: Model 1 - Train Loss: 0.049888, Valid Loss: 0.049628 | Model 2 - Train Loss: 0.059893, Valid Loss : 0.059710
Epoch 041/300: Model 1 - Train Loss: 0.048602, Valid Loss: 0.044161 | Model 2 - Train Loss: 0.060835, Valid Loss : 0.070133
Epoch 042/300: Model 1 - Train Loss: 0.049003, Valid Loss: 0.044365 | Model 2 - Train Loss: 0.059107, Valid Loss : 0.068569
Epoch 043/300: Model 1 - Train Loss: 0.048234, Valid Loss: 0.045093 | Model 2 - Train Loss: 0.058259, Valid Loss : 0.061668
Epoch 044/300: Model 1 - Train Loss: 0.046475, Valid Loss: 0.041284 | Model 2 - Train Loss: 0.057006, Valid Loss : 0.055460
Epoch 045/300: Model 1 - Train Loss: 0.045202, Valid Loss: 0.043052 | Model 2 - Train Loss: 0.055407, Valid Loss : 0.052666

Epoch 046/300: Model 1 - Train Loss: 0.044130, Valid Loss: 0.042895 | Model 2 - Train Loss: 0.054812, Valid Loss : 0.054791
Epoch 047/300: Model 1 - Train Loss: 0.044631, Valid Loss: 0.044764 | Model 2 - Train Loss: 0.052912, Valid Loss : 0.053007
Epoch 048/300: Model 1 - Train Loss: 0.042503, Valid Loss: 0.045607 | Model 2 - Train Loss: 0.052036, Valid Loss : 0.053332
Epoch 049/300: Model 1 - Train Loss: 0.043259, Valid Loss: 0.039992 | Model 2 - Train Loss: 0.050814, Valid Loss : 0.053507
Epoch 050/300: Model 1 - Train Loss: 0.041990, Valid Loss: 0.041569 | Model 2 - Train Loss: 0.049476, Valid Loss : 0.050292
Epoch 051/300: Model 1 - Train Loss: 0.040302, Valid Loss: 0.040537 | Model 2 - Train Loss: 0.048097, Valid Loss : 0.046414
Epoch 052/300: Model 1 - Train Loss: 0.039694, Valid Loss: 0.039613 | Model 2 - Train Loss: 0.049156, Valid Loss : 0.046899
Epoch 053/300: Model 1 - Train Loss: 0.039196, Valid Loss: 0.038208 | Model 2 - Train Loss: 0.048197, Valid Loss : 0.047046
Epoch 054/300: Model 1 - Train Loss: 0.038289, Valid Loss: 0.036867 | Model 2 - Train Loss: 0.046848, Valid Loss : 0.051362
Epoch 055/300: Model 1 - Train Loss: 0.037417, Valid Loss: 0.037002 | Model 2 - Train Loss: 0.045695, Valid Loss : 0.044917
Epoch 056/300: Model 1 - Train Loss: 0.037828, Valid Loss: 0.035474 | Model 2 - Train Loss: 0.045517, Valid Loss : 0.044061
Epoch 057/300: Model 1 - Train Loss: 0.037286, Valid Loss: 0.042544 | Model 2 - Train Loss: 0.044737, Valid Loss : 0.047005
Epoch 058/300: Model 1 - Train Loss: 0.037622, Valid Loss: 0.044295 | Model 2 - Train Loss: 0.043645, Valid Loss : 0.043943
Epoch 059/300: Model 1 - Train Loss: 0.037067, Valid Loss: 0.035065 | Model 2 - Train Loss: 0.044856, Valid Loss : 0.045335
Epoch 060/300: Model 1 - Train Loss: 0.037887, Valid Loss: 0.034940 | Model 2 - Train Loss: 0.042947, Valid Loss : 0.043941
Epoch 061/300: Model 1 - Train Loss: 0.037125, Valid Loss: 0.037212 | Model 2 - Train Loss: 0.042539, Valid Loss : 0.047107
Epoch 062/300: Model 1 - Train Loss: 0.035597, Valid Loss: 0.046790 | Model 2 - Train Loss: 0.041836, Valid Loss : 0.045143
Epoch 063/300: Model 1 - Train Loss: 0.037657, Valid Loss: 0.042406 | Model 2 - Train Loss: 0.042226, Valid Loss : 0.047062
Epoch 064/300: Model 1 - Train Loss: 0.035708, Valid Loss: 0.036893 | Model 2 - Train Loss: 0.041650, Valid Loss : 0.043570
Epoch 065/300: Model 1 - Train Loss: 0.035443, Valid Loss: 0.036408 | Model 2 - Train Loss: 0.041341, Valid Loss : 0.047620
Epoch 066/300: Model 1 - Train Loss: 0.034886, Valid Loss: 0.035464 | Model 2 - Train Loss: 0.041515, Valid Loss : 0.043528
Epoch 067/300: Model 1 - Train Loss: 0.036121, Valid Loss: 0.034168 | Model 2 - Train Loss: 0.040827, Valid Loss : 0.041428
Epoch 068/300: Model 1 - Train Loss: 0.035051, Valid Loss: 0.037954 | Model 2 - Train Loss: 0.042286, Valid Loss : 0.043573
Epoch 069/300: Model 1 - Train Loss: 0.035516, Valid Loss: 0.042228 | Model 2 - Train Loss: 0.040491, Valid Loss : 0.046064
Epoch 070/300: Model 1 - Train Loss: 0.034456, Valid Loss: 0.041476 | Model 2 - Train Loss: 0.040878, Valid Loss : 0.042412
Epoch 071/300: Model 1 - Train Loss: 0.034856, Valid Loss: 0.039370 | Model 2 - Train Loss: 0.040683, Valid Loss : 0.042779
Epoch 072/300: Model 1 - Train Loss: 0.033725, Valid Loss: 0.037555 | Model 2 - Train Loss: 0.041216, Valid Loss : 0.043105
Epoch 073/300: Model 1 - Train Loss: 0.034167, Valid Loss: 0.034561 | Model 2 - Train Loss: 0.039570, Valid Loss : 0.043066
Epoch 074/300: Model 1 - Train Loss: 0.034464, Valid Loss: 0.041816 | Model 2 - Train Loss: 0.039881, Valid Loss : 0.042614
Epoch 075/300: Model 1 - Train Loss: 0.033792, Valid Loss: 0.038387 | Model 2 - Train Loss: 0.039884, Valid Loss : 0.045042
Epoch 076/300: Model 1 - Train Loss: 0.033379, Valid Loss: 0.047506 | Model 2 - Train Loss: 0.039435, Valid Loss : 0.044100
Epoch 077/300: Model 1 - Train Loss: 0.033328, Valid Loss: 0.050039 | Model 2 - Train Loss: 0.039381, Valid Loss : 0.040432
Epoch 078/300: Model 1 - Train Loss: 0.033323, Valid Loss: 0.049040 | Model 2 - Train Loss: 0.038921, Valid Loss : 0.042311
Epoch 079/300: Model 1 - Train Loss: 0.033604, Valid Loss: 0.051786 | Model 2 - Train Loss: 0.039501, Valid Loss : 0.042970
Epoch 080/300: Model 1 - Train Loss: 0.033662, Valid Loss: 0.042389 | Model 2 - Train Loss: 0.039291, Valid Loss : 0.043467
Epoch 081/300: Model 1 - Train Loss: 0.034424, Valid Loss: 0.045199 | Model 2 - Train Loss: 0.038515, Valid Loss : 0.045706
Epoch 082/300: Model 1 - Train Loss: 0.033325, Valid Loss: 0.053213 | Model 2 - Train Loss: 0.040525, Valid Loss : 0.045195
Epoch 083/300: Model 1 - Train Loss: 0.033555, Valid Loss: 0.056406 | Model 2 - Train Loss: 0.038314, Valid Loss : 0.039950
Epoch 084/300: Model 1 - Train Loss: 0.034002, Valid Loss: 0.037013 | Model 2 - Train Loss: 0.042403, Valid Loss : 0.042370
Epoch 085/300: Model 1 - Train Loss: 0.034066, Valid Loss: 0.035230 | Model 2 - Train Loss: 0.041610, Valid Loss : 0.046309
Epoch 086/300: Model 1 - Train Loss: 0.035179, Valid Loss: 0.048149 | Model 2 - Train Loss: 0.039441, Valid Loss : 0.041970
Epoch 087/300: Model 1 - Train Loss: 0.032862, Valid Loss: 0.071869 | Model 2 - Train Loss: 0.038053, Valid Loss

: 0.043457
Epoch 088/300: Model 1 - Train Loss: 0.033972, Valid Loss: 0.051355 | Model 2 - Train Loss: 0.038068, Valid Loss : 0.045938
Epoch 089/300: Model 1 - Train Loss: 0.032272, Valid Loss: 0.045484 | Model 2 - Train Loss: 0.038268, Valid Loss : 0.043278
Epoch 090/300: Model 1 - Train Loss: 0.032653, Valid Loss: 0.041212 | Model 2 - Train Loss: 0.037745, Valid Loss : 0.036876
Epoch 091/300: Model 1 - Train Loss: 0.032739, Valid Loss: 0.037795 | Model 2 - Train Loss: 0.037370, Valid Loss : 0.039172
Epoch 092/300: Model 1 - Train Loss: 0.034067, Valid Loss: 0.040544 | Model 2 - Train Loss: 0.037490, Valid Loss : 0.040903
Epoch 093/300: Model 1 - Train Loss: 0.034496, Valid Loss: 0.058533 | Model 2 - Train Loss: 0.037538, Valid Loss : 0.045250
Epoch 094/300: Model 1 - Train Loss: 0.032539, Valid Loss: 0.057164 | Model 2 - Train Loss: 0.036864, Valid Loss : 0.038077
Epoch 095/300: Model 1 - Train Loss: 0.032238, Valid Loss: 0.061657 | Model 2 - Train Loss: 0.037133, Valid Loss : 0.039887
Epoch 096/300: Model 1 - Train Loss: 0.032427, Valid Loss: 0.064327 | Model 2 - Train Loss: 0.038891, Valid Loss : 0.039430
Epoch 097/300: Model 1 - Train Loss: 0.031914, Valid Loss: 0.049821 | Model 2 - Train Loss: 0.037880, Valid Loss : 0.043441
Epoch 098/300: Model 1 - Train Loss: 0.032619, Valid Loss: 0.052534 | Model 2 - Train Loss: 0.036524, Valid Loss : 0.041917
Epoch 099/300: Model 1 - Train Loss: 0.032504, Valid Loss: 0.043185 | Model 2 - Train Loss: 0.036977, Valid Loss : 0.046618
Epoch 100/300: Model 1 - Train Loss: 0.032995, Valid Loss: 0.043170 | Model 2 - Train Loss: 0.036764, Valid Loss : 0.037539
Epoch 101/300: Model 1 - Train Loss: 0.032782, Valid Loss: 0.052250 | Model 2 - Train Loss: 0.036798, Valid Loss : 0.043787
Epoch 102/300: Model 1 - Train Loss: 0.031789, Valid Loss: 0.044504 | Model 2 - Train Loss: 0.036792, Valid Loss : 0.043346
Epoch 103/300: Model 1 - Train Loss: 0.031960, Valid Loss: 0.046468 | Model 2 - Train Loss: 0.036401, Valid Loss : 0.042891
Epoch 104/300: Model 1 - Train Loss: 0.033154, Valid Loss: 0.086409 | Model 2 - Train Loss: 0.037564, Valid Loss : 0.042379
Epoch 105/300: Model 1 - Train Loss: 0.035914, Valid Loss: 0.074900 | Model 2 - Train Loss: 0.038556, Valid Loss : 0.050849
Epoch 106/300: Model 1 - Train Loss: 0.032463, Valid Loss: 0.058315 | Model 2 - Train Loss: 0.036668, Valid Loss : 0.039857
Epoch 107/300: Model 1 - Train Loss: 0.032654, Valid Loss: 0.050928 | Model 2 - Train Loss: 0.036258, Valid Loss : 0.044071
Epoch 108/300: Model 1 - Train Loss: 0.031224, Valid Loss: 0.050624 | Model 2 - Train Loss: 0.037257, Valid Loss : 0.041026
Epoch 109/300: Model 1 - Train Loss: 0.031449, Valid Loss: 0.057594 | Model 2 - Train Loss: 0.037413, Valid Loss : 0.034779
Epoch 110/300: Model 1 - Train Loss: 0.033188, Valid Loss: 0.064446 | Model 2 - Train Loss: 0.035503, Valid Loss : 0.049533
Epoch 111/300: Model 1 - Train Loss: 0.031883, Valid Loss: 0.052538 | Model 2 - Train Loss: 0.036212, Valid Loss : 0.049153
Epoch 112/300: Model 1 - Train Loss: 0.031407, Valid Loss: 0.058918 | Model 2 - Train Loss: 0.036588, Valid Loss : 0.047177
Epoch 113/300: Model 1 - Train Loss: 0.032561, Valid Loss: 0.078753 | Model 2 - Train Loss: 0.036591, Valid Loss : 0.044579
Epoch 114/300: Model 1 - Train Loss: 0.032361, Valid Loss: 0.053139 | Model 2 - Train Loss: 0.037868, Valid Loss : 0.039069
Epoch 115/300: Model 1 - Train Loss: 0.031432, Valid Loss: 0.049323 | Model 2 - Train Loss: 0.037273, Valid Loss : 0.038177
Epoch 116/300: Model 1 - Train Loss: 0.032322, Valid Loss: 0.066529 | Model 2 - Train Loss: 0.036102, Valid Loss : 0.036645
Epoch 117/300: Model 1 - Train Loss: 0.030939, Valid Loss: 0.058362 | Model 2 - Train Loss: 0.037558, Valid Loss : 0.035157
Epoch 118/300: Model 1 - Train Loss: 0.031203, Valid Loss: 0.052393 | Model 2 - Train Loss: 0.037759, Valid Loss : 0.050065
Epoch 119/300: Model 1 - Train Loss: 0.031281, Valid Loss: 0.048451 | Model 2 - Train Loss: 0.035793, Valid Loss : 0.045558
Epoch 120/300: Model 1 - Train Loss: 0.031207, Valid Loss: 0.057401 | Model 2 - Train Loss: 0.035235, Valid Loss : 0.042263
Epoch 121/300: Model 1 - Train Loss: 0.030895, Valid Loss: 0.056623 | Model 2 - Train Loss: 0.035236, Valid Loss : 0.038968
Epoch 122/300: Model 1 - Train Loss: 0.030917, Valid Loss: 0.051951 | Model 2 - Train Loss: 0.035629, Valid Loss : 0.040990
Epoch 123/300: Model 1 - Train Loss: 0.031597, Valid Loss: 0.055702 | Model 2 - Train Loss: 0.036326, Valid Loss : 0.056050
Epoch 124/300: Model 1 - Train Loss: 0.030231, Valid Loss: 0.061533 | Model 2 - Train Loss: 0.036462, Valid Loss : 0.045139
Epoch 125/300: Model 1 - Train Loss: 0.030857, Valid Loss: 0.044350 | Model 2 - Train Loss: 0.034602, Valid Loss : 0.049453
Epoch 126/300: Model 1 - Train Loss: 0.030808, Valid Loss: 0.072349 | Model 2 - Train Loss: 0.035020, Valid Loss : 0.045940
Epoch 127/300: Model 1 - Train Loss: 0.030370, Valid Loss: 0.069050 | Model 2 - Train Loss: 0.034886, Valid Loss : 0.040980
Epoch 128/300: Model 1 - Train Loss: 0.031557, Valid Loss: 0.056925 | Model 2 - Train Loss: 0.034679, Valid Loss : 0.044091

Epoch 129/300: Model 1 - Train Loss: 0.030922, Valid Loss: 0.056282 | Model 2 - Train Loss: 0.034719, Valid Loss : 0.041174
Epoch 130/300: Model 1 - Train Loss: 0.030365, Valid Loss: 0.056682 | Model 2 - Train Loss: 0.034445, Valid Loss : 0.042504
Epoch 131/300: Model 1 - Train Loss: 0.031810, Valid Loss: 0.053279 | Model 2 - Train Loss: 0.034471, Valid Loss : 0.056699
Epoch 132/300: Model 1 - Train Loss: 0.030404, Valid Loss: 0.050001 | Model 2 - Train Loss: 0.034812, Valid Loss : 0.050552
Epoch 133/300: Model 1 - Train Loss: 0.031144, Valid Loss: 0.045443 | Model 2 - Train Loss: 0.034590, Valid Loss : 0.041772
Epoch 134/300: Model 1 - Train Loss: 0.031732, Valid Loss: 0.064862 | Model 2 - Train Loss: 0.034533, Valid Loss : 0.039620
Epoch 135/300: Model 1 - Train Loss: 0.031078, Valid Loss: 0.054118 | Model 2 - Train Loss: 0.034242, Valid Loss : 0.041165
Epoch 136/300: Model 1 - Train Loss: 0.031220, Valid Loss: 0.038614 | Model 2 - Train Loss: 0.034561, Valid Loss : 0.045779
Epoch 137/300: Model 1 - Train Loss: 0.029770, Valid Loss: 0.057970 | Model 2 - Train Loss: 0.036041, Valid Loss : 0.037693
Epoch 138/300: Model 1 - Train Loss: 0.030023, Valid Loss: 0.055180 | Model 2 - Train Loss: 0.035728, Valid Loss : 0.037912
Epoch 139/300: Model 1 - Train Loss: 0.030061, Valid Loss: 0.074011 | Model 2 - Train Loss: 0.035158, Valid Loss : 0.041905
Epoch 140/300: Model 1 - Train Loss: 0.031125, Valid Loss: 0.048707 | Model 2 - Train Loss: 0.035545, Valid Loss : 0.056872
Epoch 141/300: Model 1 - Train Loss: 0.030665, Valid Loss: 0.061577 | Model 2 - Train Loss: 0.035632, Valid Loss : 0.050855
Epoch 142/300: Model 1 - Train Loss: 0.029995, Valid Loss: 0.064193 | Model 2 - Train Loss: 0.034754, Valid Loss : 0.036617
Epoch 143/300: Model 1 - Train Loss: 0.029828, Valid Loss: 0.063360 | Model 2 - Train Loss: 0.034408, Valid Loss : 0.052654
Epoch 144/300: Model 1 - Train Loss: 0.030337, Valid Loss: 0.054893 | Model 2 - Train Loss: 0.033796, Valid Loss : 0.048987
Epoch 145/300: Model 1 - Train Loss: 0.031143, Valid Loss: 0.064622 | Model 2 - Train Loss: 0.033639, Valid Loss : 0.047582
Epoch 146/300: Model 1 - Train Loss: 0.030236, Valid Loss: 0.062648 | Model 2 - Train Loss: 0.033889, Valid Loss : 0.054051
Epoch 147/300: Model 1 - Train Loss: 0.031022, Valid Loss: 0.070442 | Model 2 - Train Loss: 0.033871, Valid Loss : 0.046181
Epoch 148/300: Model 1 - Train Loss: 0.030351, Valid Loss: 0.045831 | Model 2 - Train Loss: 0.034007, Valid Loss : 0.042441
Epoch 149/300: Model 1 - Train Loss: 0.031113, Valid Loss: 0.052484 | Model 2 - Train Loss: 0.033744, Valid Loss : 0.049324
Epoch 150/300: Model 1 - Train Loss: 0.030421, Valid Loss: 0.056464 | Model 2 - Train Loss: 0.033938, Valid Loss : 0.048329
Epoch 151/300: Model 1 - Train Loss: 0.030866, Valid Loss: 0.073182 | Model 2 - Train Loss: 0.034272, Valid Loss : 0.042100
Epoch 152/300: Model 1 - Train Loss: 0.029916, Valid Loss: 0.064718 | Model 2 - Train Loss: 0.033701, Valid Loss : 0.043691
Epoch 153/300: Model 1 - Train Loss: 0.030072, Valid Loss: 0.049349 | Model 2 - Train Loss: 0.033292, Valid Loss : 0.056947
Epoch 154/300: Model 1 - Train Loss: 0.030290, Valid Loss: 0.048251 | Model 2 - Train Loss: 0.033330, Valid Loss : 0.057691
Epoch 155/300: Model 1 - Train Loss: 0.029274, Valid Loss: 0.064003 | Model 2 - Train Loss: 0.034444, Valid Loss : 0.049353
Epoch 156/300: Model 1 - Train Loss: 0.029531, Valid Loss: 0.062072 | Model 2 - Train Loss: 0.034381, Valid Loss : 0.040192
Epoch 157/300: Model 1 - Train Loss: 0.030309, Valid Loss: 0.048291 | Model 2 - Train Loss: 0.034748, Valid Loss : 0.055815
Epoch 158/300: Model 1 - Train Loss: 0.029992, Valid Loss: 0.046706 | Model 2 - Train Loss: 0.034833, Valid Loss : 0.063748
Epoch 159/300: Model 1 - Train Loss: 0.030037, Valid Loss: 0.076766 | Model 2 - Train Loss: 0.035095, Valid Loss : 0.045453
Epoch 160/300: Model 1 - Train Loss: 0.030734, Valid Loss: 0.055048 | Model 2 - Train Loss: 0.033793, Valid Loss : 0.046170
Epoch 161/300: Model 1 - Train Loss: 0.029770, Valid Loss: 0.052505 | Model 2 - Train Loss: 0.033773, Valid Loss : 0.059089
Epoch 162/300: Model 1 - Train Loss: 0.029778, Valid Loss: 0.046796 | Model 2 - Train Loss: 0.033681, Valid Loss : 0.046585
Epoch 163/300: Model 1 - Train Loss: 0.031344, Valid Loss: 0.063133 | Model 2 - Train Loss: 0.033006, Valid Loss : 0.042489
Epoch 164/300: Model 1 - Train Loss: 0.030554, Valid Loss: 0.051916 | Model 2 - Train Loss: 0.032724, Valid Loss : 0.047599
Epoch 165/300: Model 1 - Train Loss: 0.030250, Valid Loss: 0.049647 | Model 2 - Train Loss: 0.032875, Valid Loss : 0.043049
Epoch 166/300: Model 1 - Train Loss: 0.030786, Valid Loss: 0.048222 | Model 2 - Train Loss: 0.032951, Valid Loss : 0.034668
Epoch 167/300: Model 1 - Train Loss: 0.028474, Valid Loss: 0.071353 | Model 2 - Train Loss: 0.032891, Valid Loss : 0.044027
Epoch 168/300: Model 1 - Train Loss: 0.029454, Valid Loss: 0.061143 | Model 2 - Train Loss: 0.032976, Valid Loss : 0.043567
Epoch 169/300: Model 1 - Train Loss: 0.029390, Valid Loss: 0.059284 | Model 2 - Train Loss: 0.033090, Valid Loss : 0.042772
Epoch 170/300: Model 1 - Train Loss: 0.030039, Valid Loss: 0.070828 | Model 2 - Train Loss: 0.032481, Valid Loss

: 0.054374
Epoch 171/300: Model 1 - Train Loss: 0.028977, Valid Loss: 0.062418 | Model 2 - Train Loss: 0.033007, Valid Loss : 0.053550
Epoch 172/300: Model 1 - Train Loss: 0.030246, Valid Loss: 0.044367 | Model 2 - Train Loss: 0.032576, Valid Loss : 0.049719
Epoch 173/300: Model 1 - Train Loss: 0.032166, Valid Loss: 0.061718 | Model 2 - Train Loss: 0.032443, Valid Loss : 0.044297
Epoch 174/300: Model 1 - Train Loss: 0.030099, Valid Loss: 0.059082 | Model 2 - Train Loss: 0.033337, Valid Loss : 0.042221
Epoch 175/300: Model 1 - Train Loss: 0.028835, Valid Loss: 0.054144 | Model 2 - Train Loss: 0.032740, Valid Loss : 0.048826
Epoch 176/300: Model 1 - Train Loss: 0.028728, Valid Loss: 0.062223 | Model 2 - Train Loss: 0.032766, Valid Loss : 0.052158
Epoch 177/300: Model 1 - Train Loss: 0.028480, Valid Loss: 0.060330 | Model 2 - Train Loss: 0.033577, Valid Loss : 0.047837
Epoch 178/300: Model 1 - Train Loss: 0.028400, Valid Loss: 0.068063 | Model 2 - Train Loss: 0.034060, Valid Loss : 0.044762
Epoch 179/300: Model 1 - Train Loss: 0.029167, Valid Loss: 0.059477 | Model 2 - Train Loss: 0.034026, Valid Loss : 0.039140
Epoch 180/300: Model 1 - Train Loss: 0.028903, Valid Loss: 0.051821 | Model 2 - Train Loss: 0.032596, Valid Loss : 0.058784
Epoch 181/300: Model 1 - Train Loss: 0.028563, Valid Loss: 0.055950 | Model 2 - Train Loss: 0.032885, Valid Loss : 0.057169
Epoch 182/300: Model 1 - Train Loss: 0.028414, Valid Loss: 0.057866 | Model 2 - Train Loss: 0.033501, Valid Loss : 0.045521
Epoch 183/300: Model 1 - Train Loss: 0.029131, Valid Loss: 0.067496 | Model 2 - Train Loss: 0.033324, Valid Loss : 0.049634
Epoch 184/300: Model 1 - Train Loss: 0.028611, Valid Loss: 0.058072 | Model 2 - Train Loss: 0.033356, Valid Loss : 0.057059
Epoch 185/300: Model 1 - Train Loss: 0.028901, Valid Loss: 0.066932 | Model 2 - Train Loss: 0.032536, Valid Loss : 0.048501
Epoch 186/300: Model 1 - Train Loss: 0.027959, Valid Loss: 0.072600 | Model 2 - Train Loss: 0.032280, Valid Loss : 0.043874
Epoch 187/300: Model 1 - Train Loss: 0.030057, Valid Loss: 0.052163 | Model 2 - Train Loss: 0.032681, Valid Loss : 0.047658
Epoch 188/300: Model 1 - Train Loss: 0.031110, Valid Loss: 0.042245 | Model 2 - Train Loss: 0.032344, Valid Loss : 0.055165
Epoch 189/300: Model 1 - Train Loss: 0.030365, Valid Loss: 0.058708 | Model 2 - Train Loss: 0.032467, Valid Loss : 0.041182
Epoch 190/300: Model 1 - Train Loss: 0.028893, Valid Loss: 0.066624 | Model 2 - Train Loss: 0.033000, Valid Loss : 0.038280
Epoch 191/300: Model 1 - Train Loss: 0.028293, Valid Loss: 0.071115 | Model 2 - Train Loss: 0.033225, Valid Loss : 0.058716
Epoch 192/300: Model 1 - Train Loss: 0.029801, Valid Loss: 0.079988 | Model 2 - Train Loss: 0.032317, Valid Loss : 0.061123
Epoch 193/300: Model 1 - Train Loss: 0.029297, Valid Loss: 0.059827 | Model 2 - Train Loss: 0.032581, Valid Loss : 0.064064
Epoch 194/300: Model 1 - Train Loss: 0.028263, Valid Loss: 0.058835 | Model 2 - Train Loss: 0.032023, Valid Loss : 0.054103
Epoch 195/300: Model 1 - Train Loss: 0.028000, Valid Loss: 0.059198 | Model 2 - Train Loss: 0.031865, Valid Loss : 0.045323
Epoch 196/300: Model 1 - Train Loss: 0.027610, Valid Loss: 0.052211 | Model 2 - Train Loss: 0.032466, Valid Loss : 0.077439
Epoch 197/300: Model 1 - Train Loss: 0.028119, Valid Loss: 0.052665 | Model 2 - Train Loss: 0.032192, Valid Loss : 0.048829
Epoch 198/300: Model 1 - Train Loss: 0.028212, Valid Loss: 0.051661 | Model 2 - Train Loss: 0.032311, Valid Loss : 0.040758
Epoch 199/300: Model 1 - Train Loss: 0.030003, Valid Loss: 0.061807 | Model 2 - Train Loss: 0.032135, Valid Loss : 0.042824
Epoch 200/300: Model 1 - Train Loss: 0.029727, Valid Loss: 0.084675 | Model 2 - Train Loss: 0.031245, Valid Loss : 0.044739
Epoch 201/300: Model 1 - Train Loss: 0.028761, Valid Loss: 0.071807 | Model 2 - Train Loss: 0.031653, Valid Loss : 0.042951
Epoch 202/300: Model 1 - Train Loss: 0.028091, Valid Loss: 0.050256 | Model 2 - Train Loss: 0.031618, Valid Loss : 0.046697
Epoch 203/300: Model 1 - Train Loss: 0.029720, Valid Loss: 0.071130 | Model 2 - Train Loss: 0.031809, Valid Loss : 0.043537
Epoch 204/300: Model 1 - Train Loss: 0.028863, Valid Loss: 0.081528 | Model 2 - Train Loss: 0.033125, Valid Loss : 0.043229
Epoch 205/300: Model 1 - Train Loss: 0.029069, Valid Loss: 0.060425 | Model 2 - Train Loss: 0.032262, Valid Loss : 0.057751
Epoch 206/300: Model 1 - Train Loss: 0.027680, Valid Loss: 0.060010 | Model 2 - Train Loss: 0.032105, Valid Loss : 0.061335
Epoch 207/300: Model 1 - Train Loss: 0.027449, Valid Loss: 0.066487 | Model 2 - Train Loss: 0.031745, Valid Loss : 0.056658
Epoch 208/300: Model 1 - Train Loss: 0.027506, Valid Loss: 0.062914 | Model 2 - Train Loss: 0.031104, Valid Loss : 0.065737
Epoch 209/300: Model 1 - Train Loss: 0.027350, Valid Loss: 0.085117 | Model 2 - Train Loss: 0.031275, Valid Loss : 0.054626
Epoch 210/300: Model 1 - Train Loss: 0.028091, Valid Loss: 0.079247 | Model 2 - Train Loss: 0.031356, Valid Loss : 0.049895
Epoch 211/300: Model 1 - Train Loss: 0.027652, Valid Loss: 0.062950 | Model 2 - Train Loss: 0.031669, Valid Loss : 0.053699

Epoch 212/300: Model 1 - Train Loss: 0.027264, Valid Loss: 0.074009 | Model 2 - Train Loss: 0.031255, Valid Loss : 0.055479
Epoch 213/300: Model 1 - Train Loss: 0.028556, Valid Loss: 0.059814 | Model 2 - Train Loss: 0.031524, Valid Loss : 0.052097
Epoch 214/300: Model 1 - Train Loss: 0.028345, Valid Loss: 0.065434 | Model 2 - Train Loss: 0.031230, Valid Loss : 0.052142
Epoch 215/300: Model 1 - Train Loss: 0.027708, Valid Loss: 0.068150 | Model 2 - Train Loss: 0.030892, Valid Loss : 0.053722
Epoch 216/300: Model 1 - Train Loss: 0.029299, Valid Loss: 0.051720 | Model 2 - Train Loss: 0.031946, Valid Loss : 0.050479
Epoch 217/300: Model 1 - Train Loss: 0.030457, Valid Loss: 0.060113 | Model 2 - Train Loss: 0.032407, Valid Loss : 0.058181
Epoch 218/300: Model 1 - Train Loss: 0.028114, Valid Loss: 0.067676 | Model 2 - Train Loss: 0.031604, Valid Loss : 0.046840
Epoch 219/300: Model 1 - Train Loss: 0.027350, Valid Loss: 0.067938 | Model 2 - Train Loss: 0.031234, Valid Loss : 0.060685
Epoch 220/300: Model 1 - Train Loss: 0.027973, Valid Loss: 0.087816 | Model 2 - Train Loss: 0.031658, Valid Loss : 0.050191
Epoch 221/300: Model 1 - Train Loss: 0.029196, Valid Loss: 0.073157 | Model 2 - Train Loss: 0.031290, Valid Loss : 0.044720
Epoch 222/300: Model 1 - Train Loss: 0.029458, Valid Loss: 0.050404 | Model 2 - Train Loss: 0.031400, Valid Loss : 0.052325
Epoch 223/300: Model 1 - Train Loss: 0.028027, Valid Loss: 0.055804 | Model 2 - Train Loss: 0.031289, Valid Loss : 0.052891
Epoch 224/300: Model 1 - Train Loss: 0.028365, Valid Loss: 0.057077 | Model 2 - Train Loss: 0.032171, Valid Loss : 0.056312
Epoch 225/300: Model 1 - Train Loss: 0.028606, Valid Loss: 0.052218 | Model 2 - Train Loss: 0.031420, Valid Loss : 0.063988
Epoch 226/300: Model 1 - Train Loss: 0.028504, Valid Loss: 0.047915 | Model 2 - Train Loss: 0.031542, Valid Loss : 0.066337
Epoch 227/300: Model 1 - Train Loss: 0.028000, Valid Loss: 0.048720 | Model 2 - Train Loss: 0.030862, Valid Loss : 0.055914
Epoch 228/300: Model 1 - Train Loss: 0.027691, Valid Loss: 0.052254 | Model 2 - Train Loss: 0.031200, Valid Loss : 0.059464
Epoch 229/300: Model 1 - Train Loss: 0.027438, Valid Loss: 0.053135 | Model 2 - Train Loss: 0.031011, Valid Loss : 0.062516
Epoch 230/300: Model 1 - Train Loss: 0.027503, Valid Loss: 0.064457 | Model 2 - Train Loss: 0.030784, Valid Loss : 0.059847
Epoch 231/300: Model 1 - Train Loss: 0.027035, Valid Loss: 0.063522 | Model 2 - Train Loss: 0.030961, Valid Loss : 0.050286
Epoch 232/300: Model 1 - Train Loss: 0.027426, Valid Loss: 0.064768 | Model 2 - Train Loss: 0.031056, Valid Loss : 0.048547
Epoch 233/300: Model 1 - Train Loss: 0.027955, Valid Loss: 0.066451 | Model 2 - Train Loss: 0.030755, Valid Loss : 0.052301
Epoch 234/300: Model 1 - Train Loss: 0.027279, Valid Loss: 0.072429 | Model 2 - Train Loss: 0.030813, Valid Loss : 0.048213
Epoch 235/300: Model 1 - Train Loss: 0.027742, Valid Loss: 0.053190 | Model 2 - Train Loss: 0.031792, Valid Loss : 0.042261
Epoch 236/300: Model 1 - Train Loss: 0.027420, Valid Loss: 0.066134 | Model 2 - Train Loss: 0.033525, Valid Loss : 0.038360
Epoch 237/300: Model 1 - Train Loss: 0.028138, Valid Loss: 0.091555 | Model 2 - Train Loss: 0.032297, Valid Loss : 0.047648
Epoch 238/300: Model 1 - Train Loss: 0.029075, Valid Loss: 0.083637 | Model 2 - Train Loss: 0.031824, Valid Loss : 0.051275
Epoch 239/300: Model 1 - Train Loss: 0.028790, Valid Loss: 0.065189 | Model 2 - Train Loss: 0.031164, Valid Loss : 0.053370
Epoch 240/300: Model 1 - Train Loss: 0.028531, Valid Loss: 0.052716 | Model 2 - Train Loss: 0.030435, Valid Loss : 0.055704
Epoch 241/300: Model 1 - Train Loss: 0.027246, Valid Loss: 0.055981 | Model 2 - Train Loss: 0.030470, Valid Loss : 0.061737
Epoch 242/300: Model 1 - Train Loss: 0.026661, Valid Loss: 0.059143 | Model 2 - Train Loss: 0.031239, Valid Loss : 0.077942
Epoch 243/300: Model 1 - Train Loss: 0.027611, Valid Loss: 0.052236 | Model 2 - Train Loss: 0.031374, Valid Loss : 0.057278
Epoch 244/300: Model 1 - Train Loss: 0.029477, Valid Loss: 0.079172 | Model 2 - Train Loss: 0.030565, Valid Loss : 0.055780
Epoch 245/300: Model 1 - Train Loss: 0.030402, Valid Loss: 0.080891 | Model 2 - Train Loss: 0.030403, Valid Loss : 0.053909
Epoch 246/300: Model 1 - Train Loss: 0.027219, Valid Loss: 0.068212 | Model 2 - Train Loss: 0.030753, Valid Loss : 0.054540
Epoch 247/300: Model 1 - Train Loss: 0.026410, Valid Loss: 0.063453 | Model 2 - Train Loss: 0.030307, Valid Loss : 0.057729
Epoch 248/300: Model 1 - Train Loss: 0.026995, Valid Loss: 0.058514 | Model 2 - Train Loss: 0.031785, Valid Loss : 0.061122
Epoch 249/300: Model 1 - Train Loss: 0.026856, Valid Loss: 0.056201 | Model 2 - Train Loss: 0.033093, Valid Loss : 0.094759
Epoch 250/300: Model 1 - Train Loss: 0.026583, Valid Loss: 0.065810 | Model 2 - Train Loss: 0.031080, Valid Loss : 0.071797
Epoch 251/300: Model 1 - Train Loss: 0.026764, Valid Loss: 0.053807 | Model 2 - Train Loss: 0.031353, Valid Loss : 0.071507
Epoch 252/300: Model 1 - Train Loss: 0.027412, Valid Loss: 0.056104 | Model 2 - Train Loss: 0.031262, Valid Loss : 0.049454
Epoch 253/300: Model 1 - Train Loss: 0.026424, Valid Loss: 0.059307 | Model 2 - Train Loss: 0.031537, Valid Loss

: 0.052956
Epoch 254/300: Model 1 - Train Loss: 0.026700, Valid Loss: 0.064535 | Model 2 - Train Loss: 0.030900, Valid Loss : 0.062878
Epoch 255/300: Model 1 - Train Loss: 0.026110, Valid Loss: 0.069290 | Model 2 - Train Loss: 0.030230, Valid Loss : 0.063672
Epoch 256/300: Model 1 - Train Loss: 0.026338, Valid Loss: 0.075920 | Model 2 - Train Loss: 0.030404, Valid Loss : 0.057335
Epoch 257/300: Model 1 - Train Loss: 0.027813, Valid Loss: 0.074323 | Model 2 - Train Loss: 0.030269, Valid Loss : 0.055065
Epoch 258/300: Model 1 - Train Loss: 0.027027, Valid Loss: 0.069593 | Model 2 - Train Loss: 0.030024, Valid Loss : 0.062807
Epoch 259/300: Model 1 - Train Loss: 0.027209, Valid Loss: 0.057399 | Model 2 - Train Loss: 0.030166, Valid Loss : 0.069409
Epoch 260/300: Model 1 - Train Loss: 0.026767, Valid Loss: 0.052904 | Model 2 - Train Loss: 0.030146, Valid Loss : 0.067375
Epoch 261/300: Model 1 - Train Loss: 0.027490, Valid Loss: 0.059763 | Model 2 - Train Loss: 0.030175, Valid Loss : 0.066466
Epoch 262/300: Model 1 - Train Loss: 0.026387, Valid Loss: 0.063873 | Model 2 - Train Loss: 0.030864, Valid Loss : 0.062553
Epoch 263/300: Model 1 - Train Loss: 0.026472, Valid Loss: 0.071715 | Model 2 - Train Loss: 0.030010, Valid Loss : 0.073543
Epoch 264/300: Model 1 - Train Loss: 0.025976, Valid Loss: 0.069098 | Model 2 - Train Loss: 0.030678, Valid Loss : 0.057751
Epoch 265/300: Model 1 - Train Loss: 0.025812, Valid Loss: 0.068314 | Model 2 - Train Loss: 0.030092, Valid Loss : 0.054497
Epoch 266/300: Model 1 - Train Loss: 0.025892, Valid Loss: 0.060109 | Model 2 - Train Loss: 0.030032, Valid Loss : 0.063068
Epoch 267/300: Model 1 - Train Loss: 0.026716, Valid Loss: 0.068735 | Model 2 - Train Loss: 0.029972, Valid Loss : 0.062602
Epoch 268/300: Model 1 - Train Loss: 0.026124, Valid Loss: 0.064558 | Model 2 - Train Loss: 0.030044, Valid Loss : 0.061021
Epoch 269/300: Model 1 - Train Loss: 0.028224, Valid Loss: 0.054872 | Model 2 - Train Loss: 0.029733, Valid Loss : 0.065746
Epoch 270/300: Model 1 - Train Loss: 0.027234, Valid Loss: 0.059145 | Model 2 - Train Loss: 0.030271, Valid Loss : 0.080447
Epoch 271/300: Model 1 - Train Loss: 0.026503, Valid Loss: 0.071029 | Model 2 - Train Loss: 0.030902, Valid Loss : 0.082978
Epoch 272/300: Model 1 - Train Loss: 0.026361, Valid Loss: 0.072520 | Model 2 - Train Loss: 0.030453, Valid Loss : 0.056292
Epoch 273/300: Model 1 - Train Loss: 0.026959, Valid Loss: 0.070924 | Model 2 - Train Loss: 0.029896, Valid Loss : 0.065488
Epoch 274/300: Model 1 - Train Loss: 0.026123, Valid Loss: 0.077481 | Model 2 - Train Loss: 0.029958, Valid Loss : 0.068603
Epoch 275/300: Model 1 - Train Loss: 0.026322, Valid Loss: 0.062149 | Model 2 - Train Loss: 0.031023, Valid Loss : 0.073847
Epoch 276/300: Model 1 - Train Loss: 0.025984, Valid Loss: 0.072990 | Model 2 - Train Loss: 0.030738, Valid Loss : 0.073074
Epoch 277/300: Model 1 - Train Loss: 0.026280, Valid Loss: 0.050137 | Model 2 - Train Loss: 0.030401, Valid Loss : 0.055315
Epoch 278/300: Model 1 - Train Loss: 0.027377, Valid Loss: 0.069987 | Model 2 - Train Loss: 0.031262, Valid Loss : 0.073707
Epoch 279/300: Model 1 - Train Loss: 0.026367, Valid Loss: 0.058252 | Model 2 - Train Loss: 0.030623, Valid Loss : 0.076066
Epoch 280/300: Model 1 - Train Loss: 0.026001, Valid Loss: 0.057209 | Model 2 - Train Loss: 0.031685, Valid Loss : 0.075095
Epoch 281/300: Model 1 - Train Loss: 0.025762, Valid Loss: 0.051493 | Model 2 - Train Loss: 0.031475, Valid Loss : 0.046175
Epoch 282/300: Model 1 - Train Loss: 0.028300, Valid Loss: 0.065351 | Model 2 - Train Loss: 0.030428, Valid Loss : 0.059969
Epoch 283/300: Model 1 - Train Loss: 0.025918, Valid Loss: 0.069566 | Model 2 - Train Loss: 0.029998, Valid Loss : 0.064055
Epoch 284/300: Model 1 - Train Loss: 0.026437, Valid Loss: 0.074483 | Model 2 - Train Loss: 0.029852, Valid Loss : 0.055910
Epoch 285/300: Model 1 - Train Loss: 0.026451, Valid Loss: 0.066163 | Model 2 - Train Loss: 0.030157, Valid Loss : 0.069812
Epoch 286/300: Model 1 - Train Loss: 0.025734, Valid Loss: 0.070552 | Model 2 - Train Loss: 0.029811, Valid Loss : 0.061600
Epoch 287/300: Model 1 - Train Loss: 0.025711, Valid Loss: 0.072790 | Model 2 - Train Loss: 0.029633, Valid Loss : 0.054147
Epoch 288/300: Model 1 - Train Loss: 0.026002, Valid Loss: 0.069016 | Model 2 - Train Loss: 0.030238, Valid Loss : 0.052446
Epoch 289/300: Model 1 - Train Loss: 0.026792, Valid Loss: 0.076159 | Model 2 - Train Loss: 0.029809, Valid Loss : 0.064095
Epoch 290/300: Model 1 - Train Loss: 0.026189, Valid Loss: 0.054603 | Model 2 - Train Loss: 0.030105, Valid Loss : 0.056178
Epoch 291/300: Model 1 - Train Loss: 0.026347, Valid Loss: 0.065744 | Model 2 - Train Loss: 0.029934, Valid Loss : 0.057303
Epoch 292/300: Model 1 - Train Loss: 0.025624, Valid Loss: 0.067682 | Model 2 - Train Loss: 0.030274, Valid Loss : 0.058745
Epoch 293/300: Model 1 - Train Loss: 0.025678, Valid Loss: 0.063606 | Model 2 - Train Loss: 0.029487, Valid Loss : 0.067088
Epoch 294/300: Model 1 - Train Loss: 0.025761, Valid Loss: 0.072625 | Model 2 - Train Loss: 0.030195, Valid Loss : 0.052576

Epoch 295/300: Model 1 - Train Loss: 0.027938, Valid Loss: 0.057311 | Model 2 - Train Loss: 0.031123, Valid Loss : 0.068067
Epoch 296/300: Model 1 - Train Loss: 0.026332, Valid Loss: 0.066500 | Model 2 - Train Loss: 0.030462, Valid Loss : 0.080575
Epoch 297/300: Model 1 - Train Loss: 0.026018, Valid Loss: 0.080471 | Model 2 - Train Loss: 0.030331, Valid Loss : 0.075718
Epoch 298/300: Model 1 - Train Loss: 0.026332, Valid Loss: 0.068223 | Model 2 - Train Loss: 0.029480, Valid Loss : 0.067670
Epoch 299/300: Model 1 - Train Loss: 0.026294, Valid Loss: 0.062224 | Model 2 - Train Loss: 0.029897, Valid Loss : 0.054977
Epoch 300/300: Model 1 - Train Loss: 0.026303, Valid Loss: 0.056077 | Model 2 - Train Loss: 0.029846, Valid Loss : 0.059744

Summary Table

Step	train()	valid()
Forward pass	✓	✓
Compute loss	✓	✓
loss.backward()	✓	✗
optimizer.step()	✓	✗
model.train() mode	✓	✗
model.eval() mode	✗	✓
torch.no_grad()	✗	✓

Evaluate loss

Great follow-up! Let's break it down:

Why does MSE Loss ≈ 0.03 → Mean Error ≈ 0.16?

You're using **Mean Squared Error (MSE)** as your loss function. Here's what that means mathematically:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- It measures the **average of the squares** of the prediction errors.
- But what we really care about is: **how far off (on average)** are our predictions?
 - That's actually the **Root Mean Squared Error (RMSE)**:

$$\text{RMSE} = \sqrt{\text{MSE}}$$

Example:

If your model gives an MSE of **0.026**, then:

$$\text{RMSE} = \sqrt{0.026} \approx 0.161$$

That means: ➔ **On average, each prediction is about 0.16 units away from the true value.**

Why is that “pretty decent”?

If your target variable (like concrete strength) has been **scaled to range from -1 to 1**, then:

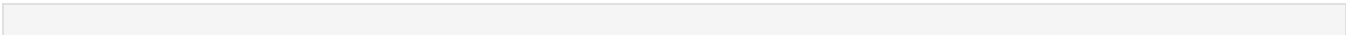
- The **total spread** of the data is only 2 units (from -1 to 1).
- A prediction error of 0.16 to 0.28 is **within 8–14% of that range**.
- That suggests your model is making **relatively accurate predictions**, especially for real-world regression tasks.

✓ Bottom Line:

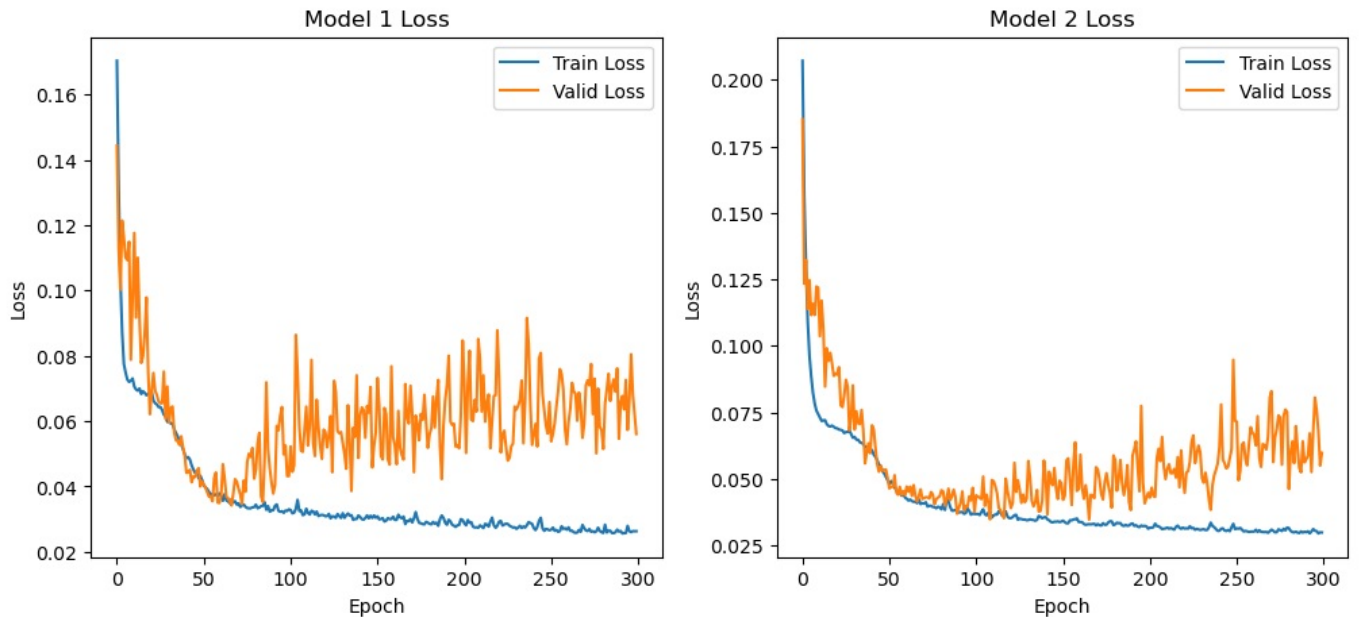
You use $\sqrt{\text{MSE}}$ to estimate **average error**, and compare that to your **target scale**. That's how we interpret “is the loss large or small.”

2.4 Please include the plot that shows how the training/validation loss vary with the training epoch. Show the plots using the above two models and write down the observation. (5%)

Plot



```
In [33]: fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].set_title("Model 1 Loss")
ax[0].plot(train_loss_1, label="Train Loss")
ax[0].plot(valid_loss_1, label="Valid Loss")
ax[1].set_title("Model 2 Loss")
ax[1].plot(train_loss_2, label="Train Loss")
ax[1].plot(valid_loss_2, label="Valid Loss")
for i in range(2):
    ax[i].set_xlabel("Epoch")
    ax[i].set_ylabel("Loss")
    ax[i].legend()
plt.show()
```



2.5 From the observation of the previous question, please determine an appropriate epoch and retrain the two models to avoid overfitting. Also, draw the loss plot of two models and save the last model as ***model1.pth*** and ***model2.pth***. (10%)

(Note. You should reload the models and the optimizer, otherwise, you will resume from the previous stop.)

Observe result and redo and save best weight

```
In [44]: epochs1 = 50
epochs2 = 80
loop1 = tqdm(range(epochs1))
loop2 = tqdm(range(epochs2))
"""
# Flexible model
model_1 = FlexibleMLP(
    input_dim=8,
    output_dim=1,
    hidden_sizes=[256, 64, 8],
    activation_fn=nn.Tanh,
    output_activation=nn.Tanh
).to(device)

model_2 = FlexibleMLP(
    input_dim=8,
    output_dim=1,
    hidden_sizes=[64, 16, 8],
    activation_fn=nn.Tanh,
    output_activation=nn.Identity # Identity, i.e., no activation
).to(device)
"""
loss_fn = nn.MSELoss()
optimizer_1 = torch.optim.SGD(model_1.parameters(), lr=0.01, momentum=0.9)
optimizer_2 = torch.optim.SGD(model_2.parameters(), lr=0.01, momentum=0.9)
train_loss_1, valid_loss_1, train_loss_2, valid_loss_2 = np.array([]), np.array([]), np.array([]), np.array([])
"""
for i in loop1:
    train_loss_1 = np.append(train_loss_1, train(train_data_loader, model_1, loss_fn, optimizer_1))
    valid_loss_1 = np.append(valid_loss_1, valid(valid_data_loader, model_1, loss_fn))
    loop1.set_description(f"Model 1 Epoch {i+1}/{epochs}")
    loop1.set_postfix(model_1_train_loss=f"{train_loss_1[-1]:.6f}", model_1_valid_loss=f"{valid_loss_1[-1]:.6f}")

for i in loop2:
    train_loss_2 = np.append(train_loss_2, train(train_data_loader, model_2, loss_fn, optimizer_2))
```

```

valid_loss_2 = np.append(valid_loss_2, valid(valid_data_loader, model_2, loss_fn))
loop2.set_description(f"Model 2 Epoch {i+1}/{epochs}")
loop2.set_postfix(model_2_train_loss=f"{train_loss_2[-1]:.6f}", model_2_valid_loss=f"{valid_loss_2[-1]:.6f}")

torch.save(model_1.state_dict(), "model1.pth") # Save the model's weights      Creates a .pth file
torch.save(model_2.state_dict(), "model2.pth") # Save the model's weights      Creates a .pth file
print("Saved PyTorch Model State to \"model1.pth\", and \"model2.pth\"")

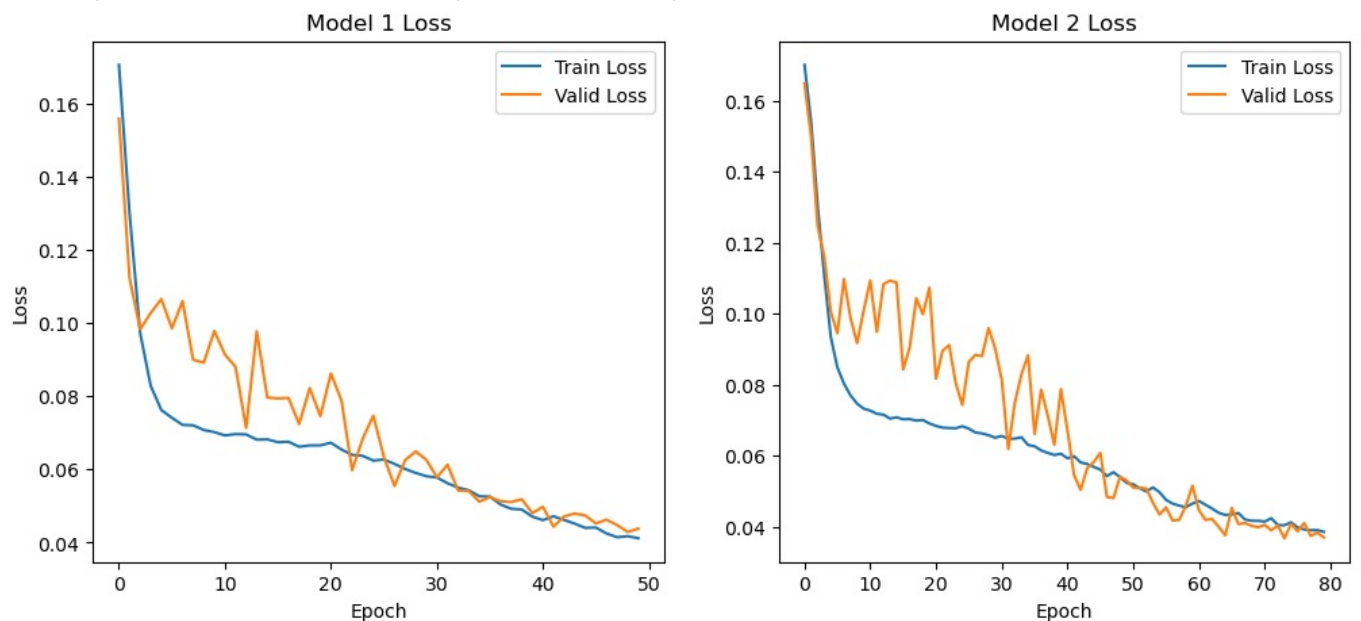
fig, ax = plt.subplots(1, 2, figsize=(12, 5))
ax[0].set_title("Model 1 Loss")
ax[0].plot(train_loss_1, label="Train Loss")
ax[0].plot(valid_loss_1, label="Valid Loss")
ax[1].set_title("Model 2 Loss")
ax[1].plot(train_loss_2, label="Train Loss")
ax[1].plot(valid_loss_2, label="Valid Loss")
for i in range(2):
    ax[i].set_xlabel("Epoch")
    ax[i].set_ylabel("Loss")
    ax[i].legend()
plt.show()

```

```

Model 1 Epoch 50/300: 100%|██████████| 50/50 [00:01<00:00, 43.61it/s, model_1_train_loss=0.041175, model_1_valid_loss=0.043793]
Model 2 Epoch 80/300: 100%|██████████| 80/80 [00:02<00:00, 27.79it/s, model_2_train_loss=0.038613, model_2_valid loss=0.037052]
Saved PyTorch Model State to "model1.pth", and "model2.pth"

```



2.6 Please load the checkpoints saved from previous question and calculate the mean squared error on test dataset for two models respectively. Also, make a dataframe with target and prediction like below and save it as ***regression.csv*** (5%)

Target	Model1	Model2
24.05	26.35	27.04
21.67	32.78	21.95

test

```

In [45]: def test(dataloader, model, loss_fn):
    model.eval()
    pred_all = np.array([])
    test_loss = 0
    with torch.no_grad():
        for X, y in dataloader:
            X, y = X.to(device), y.to(device)
            pred = model(X)
            test_loss += loss_fn(pred, y.unsqueeze(1)).item()
            pred_all = np.append(pred_all, pred.cpu().numpy()) #np.append(arr, values, axis=None)
    test_loss /= len(dataloader)
    return test_loss, pred_all

```

```

In [46]: model_1.load_state_dict(torch.load("model1.pth"))
    model_2.load_state_dict(torch.load("model2.pth"))
    model_1.eval()
    model_2.eval()

```

```

test1_loss, pred1 = test(test_data_loader, model_1, loss_fn)
test2_loss, pred2 = test(test_data_loader, model_2, loss_fn)
print(f"Test 1 average loss: {test1_loss:>8f}\nTest 2 average loss: {test2_loss:>8f}")

pred1 = scaler.y_inverse_transform(pred1)
pred2 = scaler.y_inverse_transform(pred2)

regression_result = pd.DataFrame({"Target":test_data[:, -1], "Model1": pred1, "Model2": pred2})
regression_result

```

Test 1 average loss: 0.035098
Test 2 average loss: 0.032801

C:\Users\USER\AppData\Local\Temp\ipykernel_11664\1773479504.py:1: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
model_1.load_state_dict(torch.load("model1.pth"))
```

C:\Users\USER\AppData\Local\Temp\ipykernel_11664\1773479504.py:2: FutureWarning: You are using `torch.load` with `weights\_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See <https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models> for more details). In a future release, the default value for `weights\_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add\_safe\_globals`. We recommend you start setting `weights\_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.

```
model_2.load_state_dict(torch.load("model2.pth"))
```

Out[46]:

	Target	Model1	Model2
0	26.06	36.168877	38.256497
1	10.35	15.445467	12.363482
2	74.99	55.264390	51.160890
3	59.00	52.727801	49.278089
4	52.12	42.568770	41.179509
...	...	...	...
98	33.40	35.044801	34.071871
99	39.27	35.257122	35.049937
100	18.13	20.376384	19.573824
101	33.01	23.855568	22.203571
102	33.72	36.031316	35.246158

103 rows × 3 columns

In [47]:

```
regression_result.to_csv("regression.csv", index=False)
print("Saved regression result to \"regression.csv\")
```

Saved regression result to "regression.csv"